

SEO HEE JUNG

Build it. *Validate it.* Learn it.

OS 차이 검증, 부하 테스트, 실시간 메트릭 파이프라인 구축, DevOps 인프라까지 — 직접 구현하고 검증하며 시스템을 이해한 포트폴리오

FastAPI · Linux · Ansible

Spring Boot · Resilience4j · JMeter

k6 · Kafka · Kotlin · InfluxDB

GitLab · Docker Compose · GitHub Actions · GitLab Runner · Nginx

About Me

서희정



시스템이 어떻게 동작하는지,
그리고 어떤 조건에서 한계에 도달하는지를 직접 확인하는 것을 좋아합니다.
코드가 돌아가는 것보다 **왜 돌아가는지**를 이해하는 데 집중합니다.

관심 영역

- Backend 시스템 설계 및 안정성 개선
- 관측 가능성(Observability) — 메트릭·트레이싱·로깅
- 부하 테스트 및 성능 병목 분석
- 인프라 자동화와 CI/CD 운영 환경 구축
- OS 레벨 시스템 동작 이해

프로젝트별 핵심 성과

01	OS 환경 차이 3단계 직접 검증
	Windows → Docker Linux → Native Rocky Linux 이식 및 systemctl · psutil 동작 차이 실증
02	Circuit Breaker OPEN 조건 실증
	slidingWindow 30건 · failureRate 50% 불변 설정 하에 CB 상태 전이 및 2단계 방어 체계 검증
03	Kafka consumer lag 83.2% 감소
	batch size 단일 변수 변경(500→2000)으로 병목 원인 격리 및 개선 수치 확보
04	온프레미스 DevOps 운영 흐름 구축
	Mini PC WSL2에서 GitLab · Runner · Nginx · Monitoring · Backup 기준까지 구성하고 운영 검증

공통으로 추구하는 것

"동작하는 코드"가 아닌
"왜 동작하는지 아는 코드"

직접 검증하고 · 수치로 증명하고 · 문서로 남긴다

ALL

01 / 04 · Server Monitor

FastAPI · Linux · Ansible

서버 상태 모니터링 대시보드



구현 언어 / 구성 파일 역할

Python: FastAPI 백엔드, psutil 기반 시스템 리소스·프로세스 수집**HTML/CSS/JS:** 관리자 대시보드 UI**YAML/Jinja2:** Ansible Playbook, systemd 유닛, 환경 변수 템플릿 구성

목적

개발 환경(Windows)과 운영 환경(Linux) 간 서버 모니터링 동작 차이를 직접 구현·실행·오류 경험을 통해 검증하기 위한 프로젝트

단순 수치 나열이 아닌 "이 서버에서 무엇이 실행 중이며, 지금 안전한 상태인가"를 설명하는 운영 판단형 대시보드

Windows에서 정상 동작하던 시스템 수집 로직이 Linux에서는 권한·OS API·서비스 구조 차이로 동일하게 동작하지 않음을 실증적으로 검증

개발 및 검증 프로세스 흐름

Step 1: Windows 개발 (FastAPI 대시보드)

- FastAPI를 활용하여 대시보드 기능 구현
- psutil을 사용하여 CPU, 메모리, 디스크, 프로세스 정보에 대한 검증 수행

Step 2: Docker Rocky Linux

- 동일한 코드를 Docker Rocky Linux 환경에서 실행하여 1차적인 환경 이식성 확인

Step 3: OS 간 차이점 분석 (Docker Rocky Linux와 Windows 환경)

- psutil 권한 영향 차이
- systemctl 호출 가능 여부
- 네트워크/포트 권한 제한
- 로그·리소스 접근 방식 차이

Step 4: Native Rocky Linux (최종 전체 검증)

- systemd, 로그, 권한 설정을 포함한 전체 시스템에 대해 종합적으로 검증 완료

주요 기능 4가지

① 시스템 리소스 수집

- CPU / Memory / Disk / Uptime 실시간 조회
- psutil 기반 OS 레벨 정보 수집

② 서비스 상태 모니터링

- systemctl 기반 서비스 상태 조회 (Linux)
- active/inactive/failed 판단
- Windows는 일부 제한

③ 프로세스 분석

- 실행 중 프로세스 목록 · PID · 실행 경로 · CPU/메모리 · 실행 유저 · 열린 포트 수집 및 시각화

④ 정책 기반 해석

- MongoDB/JSON KNOWN_PROCESSES 정책으로 정상/일반/의심 분류. 종료 기능 + 인증/인가 포함

위험 판단 자동화 기준

- root/관리자 권한으로 실행 중인 프로세스
- 주요 공개 포트 및 시스템 포트 사용 여부
- 과도한 메모리 사용 프로세스
- 비정상 실행 경로 (tmp, 숨김 디렉터리 등)
- KNOWN_PROCESSES 미등록 프로세스 식별 및 플래깅

환경별 서비스 감지 방식

환경	감지 방식	상태 예시
Rocky Linux (Host)	systemctl	active / failed
Docker (Container)	psutil	active / zombie
Windows	platform 체크	not supported

완료 / 예정 현황

- ✓ CPU/RAM/Disk/Uptime 시각화 — 1차 완료
- ✓ 서비스 상태 + 로그 tail — 1차 완료
- ✓ 프로세스 위험 분석 + 포트 보안 경고 — 2차 완료
- ✓ MongoDB 연동 + 프로세스 종료 기능 — 3차 완료
- ✓ 인증/인가 적용 + Ansible 운영 자동화 — 완료
- ✓ Rocky Linux Native 최종 검증 — 완료
- 🕒 [향후 확장] 다중 서버 관리 / Email 알림 / 관리자 권한 분리

로그 모니터링

journalctl -u app / nginx / docker / ssh

최근 N줄 기준 조회 (journalctl -n)

콘솔 스타일 Retro UI 표시

Ansible 기반 운영 환경 재현

서비스 배포 도구가 아닌 **운영 환경 재현** 도구로 사용. "서버 1대 재설치 → 동일 상태 복구"가 목적

Ansible 수행 핵심 작업 흐름

STEP 01 : Git Clone

/home/rockylinux/server-monitor/ 경로로 코드 다운로드

STEP 02 : 운영 디렉터리 세팅

/opt/server-monitor/ 생성 후 web/app/ 복사 배치

STEP 03 : Python venv 환경 구성

/opt/server-monitor/venv/ 생성 + requirements.txt 패키지 설치

STEP 04 : systemd 서비스 등록

FastAPI 앱 systemd 유닛 배치 + 자동 시작 기반 구성

STEP 05 : MongoDB 인증 + Vault 비밀 값

MongoDB 인증 활성화 + Ansible Vault 기반 민감 정보 분리

디렉토리 역할 분리

/home/rockylinux/server-monitor/

└ Git 레포 전체 clone

/opt/server-monitor/ ← 운영 전용

└ app/ Ansible이 web/app 배치

└ venv/

└ logs/

└ data/

ANSIBLE 관여 범위 / 비관여 범위

영역	관여 여부	영역	관여 여부
패키지 · 유저 · 권한	✅ Ansible 담당	FastAPI 내부 로직	앱 책임 — 미 관여
방화벽 포트 · SSH 보안	✅ Ansible 담당	psutil 분석 · 위험 판단	코드+DB 책임 — 미 관여
systemd 서비스 등록	✅ Ansible 담당	UI / 템플릿	프론트 책임 — 미 관여

전체 아키텍처

서버 (psutil / systemctl / 로그 / 프로세스)



FastAPI 백엔드

↓ JSON / Jinja2 Template

웹 대시보드 (Retro Console UI)



Nginx Reverse Proxy

PYTHON VENV 운영 정책

개발(Windows) : venv 수동 활성화

운영(Linux) : systemd에서 venv python 직접 지정 → Ansible이 systemd 유닛 자동 등록

OS 차이에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : psutil 프로세스 수집 권한 차이

Windows에서는 정상 수집되던 프로세스 정보가 Linux에서 PermissionError 발생.
root 권한 없이 타 유저 프로세스의 open files, connections 수집 불가.
→ 예외 처리 분기 + 권한별 수집 가능 필드 명시적 분리로 해결

ISSUE 02 : systemctl — Docker 컨테이너에서 미 동작

Docker 컨테이너 내부에는 systemd가 없어 systemctl 호출 시 Failed to connect to bus 오류.
→ 환경 감지 후 컨테이너에서는 psutil 프로세스 검색 방식으로 폴백, Native Linux에서만 systemctl 사용

ISSUE 03 : MongoDB 인증 활성화 후 연결 실패

Ansible로 MongoDB 인증 활성화 적용 후 기존 연결 문자열 모두 인증 오류.
→ Ansible Vault로 username/password 분리 관리, 환경 변수 주입 방식으로 하드코딩 제거

핵심 학습 포인트

- **같은 코드, 다른 동작**
OS API, 권한 모델, 서비스 구조가 달라지면 동일 코드가 전혀 다르게 동작함을 직접 검증
- **단계별 이식 검증의 중요성**
Windows → Docker Linux → Native Linux 순으로 차이를 단계별 격리해야 원인 특징이 가능
- **운영 자동화는 재현성**
Ansible은 배포 도구가 아니라 "서버 1대 재설치 후 동일 상태 복구"를 위한 재현 도구로 설계
- **모니터링의 목적**
수치 나열이 아닌 운영자가 판단을 내릴 수 있는 해석 정보 제공이 핵심

개선 예정 / 확장 목표

1. 다중 서버 관리 — 단일 대시보드에서 N대 관리
2. Email 알림 — 위험 이벤트 기반 자동 발송
3. 프로세스 등록 Form — 중복 빈도(cnt) 기반 자동 분류
4. 권한 분리 — Viewer / Operator / Admin 롤
5. 종료 사유 세분화 — 권장 조치 메시지 고도화

프로세스 분류 체계 요약

분류	기준
정상 프로세스	KNOWN_PROCESSES 등록 + 정상 경로
일반 서비스	등록되었으나 모니터링 대상
의심 프로세스	미등록 or 위험 패턴 감지

01 / 04 · Server Monitor — 인증 설계 & 코드 구조

관리자 전용 인증 시스템

일반 사용자용 서비스가 아닌 관리자 전용 서버 모니터링 도구
인터넷 공개 서비스가 아닌 내부 관리자 도구 전제로 설계

인증 흐름

클라이언트 → **X-API-Key** 헤더 → /api/auth

서버: 환경 변수 ADMIN_API_KEY와 비교

↓ 인증 성공 시

admin_session 쿠키 발급 (HttpOnly)

↓ 이후 요청

API Key 없이 쿠키 기반 인증 유지

JWT 미사용 이유

- 외부 사용자 인증 시스템 아님 — 토큰 갱신·서명 검증·만료 관리 불필요
- 단일 관리자 환경에서 오버 엔지니어링
- 서버 메모리 기반 세션 구조 선택

보호 대상 API

/api/dashboard : 대시보드 데이터

/api/process/terminate : 프로세스 종료

/api/admin/* : 관리 기능

모든 요청: **admin_session** 쿠키 검증 필수

실제 코드 구조 (WEB/APP/)

main.py 단일 엔트리 포인트

routes/

- └ auth.py 인증/세션
- └ dashboard.py 리소스 수집 API
- └ process.py 프로세스 분석/종료
- └ admin.py 관리 기능

services/system/

- └ cpu.py / memory.py / disk.py
- └ process_analyzer.py 위험 판단
- └ process_control.py 종료 제어
- └ service.py systemctl 연동
- └ log.py journalctl 조회

constants/

- └ linux.py / windows.py / ports.py

프로세스 분석 핵심 코드 포인트

collect_processes()

- └ CPU 측정 초기화 후 0.1s sleep
- └ process_iter(attrs=[pid, name, exe, username])
- └ oneshot()으로 한 번에 데이터 수집
- └ PID 0 무조건 제외
- └ 컨테이너 환경 ignore 프로세스 제외
- └ AccessDenied → cpu/memory None 처리

PSUTIL ONESHOT() 선택 이유

proc.oneshot()으로 내부 데이터를 한 번의 시스템 콜로 수집.

프로세스별 cpu_percent / memory_percent / connections를 개별 호출하면 각각 시스템 콜 발생 → 성능 저하

위험 판단 기준 & 종료 정책 상세

Warning Rules (위험 레벨 기준)

Rule	조건	레벨
RUNNING_AS_ADMIN	root/SYSTEM/Administrator 실행 (PID 4 제외)	 WARN
PUBLIC_PORT(n)	KNOWN_PORTS 주요 서비스 포트 사용	 / 
SYSTEM_PORT(n)	port < 1024 (KNOWN_PORTS 제외)	 / 
HIGH_MEMORY_USAGE	메모리 사용률 ≥ 20%	 INFO
SUSPICIOUS_PATH	표준 경로 외 실행 (치명 Warning)	 DANGER

치명 Warning (SUSPICIOUS_PATH, RUNNING_AS_ADMIN) 하나라도 존재 → DANGER 판정

상태 판정 기준

상태	조건
OK	KNOWN_PROCESSES 등록 + Warning 없음
WARN	UNKNOWN or 일반 Warning 존재
DANGER	치명 Warning 하나라도 존재

종료 차단 대상

OS	차단 대상
Windows	SESSION 0, SYSTEM/LOCAL/NETWORK SERVICE, Windows signed core binary
Linux	UID 0 (root), systemd/daemon, /usr/lib/systemd

종료 판단 플로우

- [사용자 종료 요청]
- ↓ 실시간 psutil 재수집
- ↓ MongoDB known_processes 조회 (연결 시)
- ↓ 시스템 프로세스 여부 확인
- ↓ Soft Kill (SIGTERM, 최대 5초 대기)
- ↓ Hard Kill (SIGKILL, 실패 시만)

MongoDB 역할 분리

- MongoDB는 종료 판단의 **필수 의존성이 아님** : 연결 실패 시에도 종료 기능 정상 동작
- MongoDB : 이름 기반 정적 정책 저장소 (policy.is_system, policy.terminatable)
- 실시간 psutil 판단이 항상 최종 기준 : MongoDB가 시스템 보호 우회 불가
- UI용 종료 사유 텍스트(policy.reason) 제공

02 / 04 · Circuit Breaker Tester

Spring Boot · Resilience4j · JMeter

부하 대응 및 관측 시스템

 GitHub

구현 언어 / 구성 파일 역할

Java : Spring Boot 기반 부하 재현 API, Rate Limit, Circuit Breaker, Metrics 구현**YAML** : Spring 설정, Docker Compose, Prometheus 구성**SQL** : PostgreSQL 테스트 데이터 조회 및 DB 상태 검증

테스트 / 관측 도구

Jmeter : 외부 레포 기반 비정상 트래픽 생성 (jmeter-traffic-generator)**Prometheus** : Micrometer 기반 메트릭 수집 및 노출

목적

정상 운영이 아닌 극한 상황에서 시스템이 어떻게 반응하는지를 정밀 계측.

CPU 포화·DB 커넥션 고갈 등 장애를 API로 재현하고 Rate Limit·Circuit Breaker 트리거를 실증 검증.

성능 최적화가 아닌, 시스템이 한계에 도달하기 직전 어떤 일이 벌어지는지를 관측하기 위한 테스트 전용 백엔드.

개발 및 검증 프로세스 흐름

1. Vault로 DB 자격 증명 주입
2. Docker로 로컬/운영 차이 제거
3. 부하 생성·방어 로직 Service 계층 집중
 - ↳ AOP로 관측 로직 분리
4. TraceIdFilter → trace_id 전파
 - ↳ Filter → Service → DB 전 구간
5. jmeter-traffic-generator 외부 레포 기반 비정상 트래픽 주입
6. API + 부하로 CB OPEN 상황 유도
7. Prometheus + Snapshot API 검증

2단계 방어 체계

요청 인입

↓

1차 방어 : IP 기반 Rate Limit

↳ 반복 트래픽 차단 (X-Forwarded-For 기반)

↓ 통과 시

2차 방어 : Resilience4j CircuitBreaker

↳ CLOSED → HALF_OPEN → OPEN 상태 전이

↳ slidingWindowSize:30, failureRate:50%

↳ 15건 실패 시 OPEN

테스트 불변 설정

50%

CB failureRate
Threshold

30

CB sliding
WindowSize

50

HikariCP
maxPoolSize

5s

Connection
Timeout

JMeter 트래픽 구성

JMeter 시나리오는 별도 **jmeter-traffic-generator** 저장소 기반으로 구성
본 백엔드는 부하를 수신하고 Rate Limit / Circuit Breaker / Metrics를 검증하는 대상 서버 역할 수행

Attack vs Normal 트래픽 분리

항목	Attack-TG	Normal-TG
목적	부하 유발	가용성 확인
Thread 수	300→500→750→1000	25
Delay	100~300ms	1000ms
IP (X-Forwarded-For)	10.10.10.10	20.20.20.20

실험 제약 조건

- **Concurrency Stress Test** : Thread 수만 단계적 증가, ON/OFF 비교는 동일 단계에서만
- **Resource Exhaustion Test** : repeatCount 증가, Delay 0ms, Pool Size 축소 허용
- 두 실험은 직접 비교하지 않음 : 목적 상이
- CSV 최소 필드 저장 + HTML 리포트 생성으로 분석

부하 시나리오

CPU Load

Busy-loop 연산으로
실제 CPU 점유율 상승
유도

DB Read

SELECT 반복 →
커넥션 풀 고갈 · 지연
재현. 점진적 병목

DB Write

INSERT 반복 → 락
경합 · Commit 비용
증가. 급격한 병목

Cache ON/OFF

Redis 캐시 적용
여부에 따른 DB 부하
변동 실시간 관측

시나리오 비교

	DB READ	DB WRITE
API	/db-read	/db-write
병목 위치	Connection Pool	Lock / Commit
장애 양상	점진적 지연	급격한 병목

JMeter 아키텍처

JMeter (독립 컨테이너)
↓ Docker bridge network
Nginx L7 Reverse Proxy
↓
Backend (Spring Boot)
└ DB · Redis · Metrics

Observability 설계

trace_id 기반 요청 추적

모든 요청에 고유 **trace_id** 부여 (TraceldFilter)

↓ MDC를 통해 로그 전파

Filter → Service → Repository 전 구간 추적

↓ 응답 헤더까지 일관 전파

AOP 기반 로깅 아키텍처

LogAspect : 단일 진입점 관측 로깅

이벤트 체계 : LOAD_START / END / FAIL / ERROR

JSON Logback 구조화 로그 (trace_id · level · duration) → Wazuh 연계 전제로 설계

Custom Metrics (Prometheus)

- rate_limit_blocked_total : Rate Limit 차단 건수
- rate_limit_allowed_total : 허용 건수
- resilience4j.circuitbreaker.* : CB 상태
- hikaricp.connections.* : 풀 가용량
- http.server.requests : 요청 시계열

기술 스택 상세

분류	기술
Language	Java 17 (Eclipse Temurin Alpine)
Framework	Spring Boot 3.4.2
Resilience	Resilience4j (Count-based CB)
RDBMS	PostgreSQL 16
Cache	Caffeine L1 + Redis 7 L2
Metrics	Micrometer + Prometheus
Tracing	MDC + TraceldFilter
Secret	Spring Cloud Vault / HashiCorp Vault (AppRole)
Deploy	Docker Multi-stage build

완료 / 예정 현황

- ✅ CB CLOSED/HALF_OPEN/OPEN 전이 확인
- ✅ IP Rate Limit 1차 방어
- ✅ CPU / DB READ / DB WRITE 부하 API
- ✅ SystemSnapshot / RecentRequests API
- ✅ Prometheus + Actuator 연동
- ✅ Vault KV(v2) DB 자격 증명 외부화
- ✅ Graceful Shutdown (SIGTERM 안전 종료)
- ⌚ [향후 확장] Spring Security + 테스트 API 보호
- ⌚ [향후 확장] spring_logs 테이블 실제 저장 로직 연결

부하 상황에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : CB가 예상 시점보다 늦게 OPEN

slidingWindowSize=30 기준 15건 실패 시 OPEN 예상.

실제로는 HikariCP connectionTimeout(5s)이 먼저 걸려 slow call로 집계되는 경우가 섞이면서 failureRate 집계가 지연됨.

→ CB 실패 정의(exception vs slow call)와 풀 타임아웃 타이밍을 명확히 분리해 설정

ISSUE 02 : Normal-TG까지 Rate Limit에 걸림

Attack-TG와 Normal-TG가 같은 Docker bridge를 사용해 실제 소스 IP가 동일하게 보임.

→ X-Forwarded-For 헤더로 IP를 명시 분리(10.10.10.10 / 20.20.20.20) 하고 Rate Limit 로직을 해당 헤더 기반으로 수정

ISSUE 03 : Vault 연동 후 앱 시작 시 자격 증명 로드 실패

application.yml은 AppRole 인증(authentication: APPROLE) 사용.

setup.md는 로컬 실행용 Token 방식 안내로 코드와 혼재.

→ 포트폴리오 기준: AppRole 설정 사용, 로컬 개발은 Token 방식 병행 확인 필요로 문서화

핵심 학습 포인트

- 장애는 설계로 제어한다

문제가 생긴 뒤 대응이 아니라 임계치를 수치로 정의하고 의도적으로 트리거 해야 검증 가능

- 불변 설정의 중요성

실험 중 설정값 변경 시 어떤 변수가 결과를 바꿨는지 분리 불가.

CB/HikariCP/Redis TTL을 고정해야 재현성 확보

- trace_id 전파의 실용성

장애 발생 시 Filter → Service → DB 구간 지연을 trace_id 하나로 추적 가능.

실제 장애 분석 시간 단축에 직결

- 2단계 방어의 의미

Rate Limit(IP 기반)이 없으면 Normal-TG까지 CB에 걸려 정상 사용자도 차단됨. 계층별 방어가 필수

CB 상태 전이 실증

CLOSED : 정상 운영 (요청 통과)

↓ 30건 중 15건(50%) 실패 시

OPEN : 즉시 차단 (fallback 반환)

↓ waitDuration 경과 후

HALF_OPEN : 제한적 허용 (복구 탐색)

↓ 성공 시 CLOSED / 실패 시 OPEN

향후 확장

1. Spring Security 도입 : 테스트 API 오남용 방지
2. spring_logs 테이블 실제 저장 로직 연결
3. Wazuh 연계 : JSON 로그 기반 보안 이벤트 탐지
4. Vault dynamic secrets 도입 검토 (현재 AppRole 설정 사용)

TraceId 전파 & AOP 로깅 설계

TraceIdFilter — 실제 코드 핵심

```
// 외부 전달 시 재사용, 없으면 신규 생성
String traceId = Optional.ofNullable(request.getHeader("X-Trace-Id"))
    .orElse(UUID.randomUUID().toString());

MDC.put	TRACE_ID_KEY, traceId);
response.setHeader("X-Trace-Id", traceId);

try { filterChain.doFilter(...); }
finally { MDC.clear(); } // 스레드 오염 방지
```

LogAspect — 적용 범위 & 이벤트

```
@Around(
    "execution(* LoadScenarioService.*(..))
    || execution(* CircuitBreakerTestService.*(..))"
)

LOAD_START → log.info(event, method)
LOAD_END → log.info(event, method, durationMs)
LOAD_FAIL → log.warn (비치명적, CB OPEN 등)
LOAD_ERROR → log.error (치명적 실패)
```

RateLimitFilter — 실제 구현 포인트

Bucket4j Token Bucket 방식
 IP별 Bucket → Caffeine Cache 저장
 expireAfterAccess 10min, max 10,000 IP

런타임 ON/OFF (서버 재시작 없이)

SystemHealthService.toggleRateLimit()

차단 시 : **rate_limit_blocked_total** 증가
 허용 시 : **rate_limit_allowed_total** 증가
 차단 요청도 **trace_id** 포함

LogLevelPolicy — 예외 유형별 레벨

CallNotPermittedException (CB OPEN) → WARN (비치명 — 설계된 차단)

DB / I/O Exception → ERROR (치명 — 인프라 문제)

ApiException (비즈니스 예외) → BUSINESS_EXCEPTION 이벤트

Filter 실행 순서

1st TraceIdFilter → trace_id 생성
2nd RateLimitFilter → 차단/허용
3rd Controller → Service → Repository
AOP Resilience4j CB (Service 감싸기)

RuntimeFeatureState — 토크 저장소

- RateLimit ON/OFF 런타임 상태 저장
- Redis Cache ON/OFF 런타임 상태 저장
- 서버 재시작 시 application.yml 기준 초기화
- SystemSnapshot에 현재 상태 반영

03 / 04 · k6 Realtime Metrics Pipeline

k6 · Kafka · Kotlin · InfluxDB

스트리밍 기반 부하 테스트 파이프라인



구현 언어 / 구성 파일 역할

Kotlin : Kafka Consumer 기반 Collector, InfluxDB line protocol 변환 · 적재**JavaScript** : k6 부하 테스트 시나리오 작성 (smoke / load / stress)**SQL** : InfluxDB v3 / Grafana 데이터 조회 쿼리 작성**YAML** : Kafka, InfluxDB, Grafana Docker Compose 구성

목적

부하 테스트 메트릭이 생성되는 순간부터 수집 · 저장 · 시각화 · 병목 분석까지 이어지는 관측 파이프라인을 직접 구축.

서비스 경로와 메트릭 처리 경로를 분리해 독립적으로 분석.

전체 아키텍처

[Load Path]

k6 — HTTP Load —▶ Kotlin API

[Metrics Path]

k6 → Kafka (k6-metrics) → Kotlin Collector
→ InfluxDB v3 → Grafana

[Infra Path — 향후 확장 계획]

Terraform → Azure 리소스 생성
Ansible → VM 환경 구성

설계 의도 — 경로 분리

서비스 트래픽 k6 → Kotlin API

↳ latency / error / status 결정

메트릭 처리 k6 → Kafka → Collector

↳ 성능 측정 대상 아님

↳ 파이프라인 자체 상태 관측

k6는 기본적으로 InfluxDB v3 직접 연동 미지원
→ xk6-output-kafka 커스텀 빌드로 Kafka 경유
스트리밍 구조 설계

Kafka 도입 이유

- producer(k6) / consumer(Collector) 비동기 분리
- burst traffic 완충 : 장애 격리 · 재처리 여지
- 실시간 스트림 처리 구조 확보

Kotlin Collector 도입 이유

- 저장 전 필드 정규화 레이어 확보
- URL full string → path 기준 정규화
- batch write 최적화 및 실패 시 재시도
- high cardinality tag 사용 제한

Grafana 시각화 지표

- RPS (초당 요청 수) : 부하 정상 유입 확인
- Response Time (avg / p95) : 지연 급증 구간
- Failed Requests : 장애 발생 시점
- VUs : 시나리오 반영 여부
- Collector TPS · buffer_size · last_flush_ms

※ kafka_lag는 Kafka consumer group CLI 기준 별도 확인

Collector → InfluxDB 병목 분석 및 개선

개선은 한 번에 하나만 적용. 여러 설정 동시 변경 시 어느 변경이 효과를 냈는지 분리 불가

문제 상황 (stress 시나리오)

stress 조건 : 100 → 200 → 500 VUs
Collector batch size : 500
batch write : 70KB~75KB / flush: ~990ms

k6 → Kafka 유입 정상

Collector consume 정상
InfluxDB write status : 204 (성공)

그러나 **Kafka consumer lag 대량 증가** → 메시지 유입량 대비 write 처리량 부족

개선 적용

- 변경 : INFLUX_BATCH_SIZE 500 → 2000
- └ InfluxDB write 호출 횟수 감소
 - └ Collector batch 처리 효율 개선
 - └ Kafka backlog 처리 지연 완화

Before / After 수치 비교

83.2%

Kafka consumer lag 감소율

4x

batch size 증가
500 → 2000

지표	Before	After
Kafka lag	1,212,689	203,648
avg latency	4~5ms	~3.2ms
RPS	~10	~10
Failed Req	0%	0%
Response Time	Grafana 지연	정상 표시

개선을 계산

lag 감소율 = $(1,212,689 - 203,648) / 1,212,689 \times 100$
= 83.2%

최종 판단

병목 위치 : Collector → InfluxDB write
개선 방법 : INFLUX_BATCH_SIZE 조정
load 기준 RPS · error · lag 안정 유지
stress 기준 lag 83.2% 감소 확인

주요 컴포넌트 역할

부하 시나리오

smoke

연결 상태 확인. 최소 부하

load

동일 조건 기준 성능 측정.
Before/After 비교

stress

VUs 단계 증가. 병목 탐색
전용

Kotlin Collector 처리 전략

- Kafka 메시지 poll 단위로 수신
- InfluxDB write는 batch 단위 (시간 기준 1초 or 건수 기준)
- batch 처리량 / write 결과 / tps를 내부 메트릭으로 InfluxDB 적재
(kafka_lag는 CLI 기준 별도 확인)
- write 실패 시 commitSync() 미호출 → Kafka가 동일 메시지 재전달 (At-Least-Once)
- URL path 정규화 + high cardinality tag 제한

InfluxDB Schema

Tag: scenario / method / status / metric_name / group

Field: duration / count / value / vus

URL full string → **path** 기준 정규화

Collector 내부 상태 검증 결과

buffer_size = 0 (flush 정상)

failed_total = 0 (처리 실패 없음)

kafka_lag = 0 (placeholder — CLI로 별도 확인)

last_flush_ms = ~485ms (write 지연 없음)

processed_total = 4302+ (지속 증가)

tps = 2.13~2.19

완료 / 예정 현황

- ✓ k6 → Kafka 연결 (xk6-output-kafka 빌드)
- ✓ Kotlin Collector consume → InfluxDB write
- ✓ InfluxDB v3 measurement 생성 확인
- ✓ SQL 기반 데이터 조회 성공
- ✓ Grafana provisioning (datasource/dashboard)
- ✓ 병목 개선 사례 1건 확보
- ⌚ [향후 확장] Azure Terraform (VM, VNet, NSG)
- ⌚ [향후 확장] Azure Ansible (Docker 설치, 서비스 기동)

Kafka listener 설정 핵심

- 수동 producer/consumer 성공 ≠ k6 연결 성공
- k6는 별도 컨테이너 → broker 주소 해석 다름
- advertised.listeners 분리
- PLAINTEXT://kafka:9092 (내부)
- PLAINTEXT_HOST://host.docker.internal:29092 (외부)

파이프라인 구축에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : 수동 Kafka 테스트 성공 ≠ k6 연결 성공

컨테이너 내부에서 kafka-console-producer로 메시지 전송은 성공.
그러나 k6 컨테이너(별도 네트워크)에서는 broker 주소 해석이 달라 연결 실패.
→ advertised.listeners를 PLAINTEXT(내부)/PLAINTEXT_HOST (외부 host.docker.internal:29092)로 분리해 해결

ISSUE 02 : InfluxDB 토큰 down -v 후 전체 무효화

docker compose down -v로 볼륨 초기화 시 기존 토큰 전부 삭제.
Collector와 Grafana는 이전 토큰을 계속 사용해 인증 실패. → 토큰 생성 → .env 반영 → 전체 재기동을 하나의 절차로 문서화.
provisioning secureJsonData에서 환경 변수 치환 미동작 문제도 동일 원인

ISSUE 03 : Grafana dashboard datasource UID 불일치

UI에서 export한 dashboard JSON이 기존 datasource ID를 참조.
provisioning으로 새로 생성된 datasource와 UID가 달라 패널 데이터 미표시.
→ dashboard JSON의 datasource 참조를 provisioning datasource 이름 기준으로 수정

핵심 학습 포인트

- **파이프라인은 경로 분리가 핵심**
서비스 트래픽과 메트릭 처리를 같은 경로에 두면 장애 원인 격리가 불가능. 분리 설계가 관측의 전제
- **병목은 데이터로 증명**
Kafka consumer group CLI와 Collector 내부 지표를 함께 확인해 병목 위치를 특정하고, 단일 변수(batch size)만 변경해 개선 효과를 수치로 확보
- **Kafka는 완충재**
burst traffic 시 Collector가 처리 속도를 따라가지 못해도 메시지는 유실되지 않고 Kafka lag로 적체. 이후 Consumer가 순차적으로 따라잡을 수 있는 재처리 가능한 구조
- **InfluxDB v3 직접 write**
공식 Kotlin client(v2 기반) 미사용, /api/v3/write_lp HTTP 직접 호출로 의존성 최소화

병목 탐색 프로세스

1. **탐색** : stress 실행 후 Grafana/CLI로 lag 확인
2. **가설** : Collector → InfluxDB write 처리량 부족
3. **격리** : 단일 변수(batch size)만 변경
4. **재측정** : 동일 stress 조건 재실행
5. **증명** : lag 83.2% 감소 수치 확보

향후 확장

1. Azure Terraform → VM, VNet, NSG 자동 생성
2. Azure Ansible → Docker 설치, 서비스 배포 자동화
3. p95/p99 latency 별도 Grafana 패널 확장

KafkaConsumer.kt 핵심 설계

Offset Commit 전략 (코드 기반)

enable.auto.commit = false

```
for (record in records) {
    val line = processor.toLineProtocol(rawMessage)
    val success = writer.add(line)
    if (!success) {
        allSucceeded = false
        break // write 실패 시 루프 탈출
    }
}

if (allSucceeded) {
    consumer.commitSync() // write 성공 후만
}
```

※ write 실패 시 commitSync() 미호출 → Kafka가 동일 메시지 재전달 → At-Least-Once 보장

장애 시나리오별 동작

시나리오	동작
Collector 중단	재기동 후 earliest부터 재처리
InfluxDB 중단	retry 후 dead-letter 로그, offset commit 미호출
JSON 파싱 실패	해당 메시지 skip, 실패 카운트 증가
k6 burst	Kafka 버퍼, batch 단위 처리

MetricsProcessor — 정규화 규칙

metric : 공백 제거

tag : 공백 → _ / null 제거

timestamp : ISO → nanosecond

value : Double, NaN/Inf 제외

url query string 제거 : /user/123 → /user/:id 정규화

Batch Flush 조건 (2개 중 먼저)

500건

초기 batch-size
(→ 2000으로 개선)

1000ms

flush interval
(시간 기준)

재시도 정책

- max-retries: 3
- backoff: 1s → 2s → 4s (지수 백오프)
- 대상: HTTP 429 / 5xx
- 제외: 400 / 401 / 403
- 최종 실패: dead-letter 파일 기록

collector_stats measurement

processed_total 처리 건수

failed_total 실패 건수

tps 초당 처리량

buffer_size 현재 버퍼 크기

last_flush_ms flush 소요 시간

→ InfluxDB에 동일 경로로 저장

→ Grafana Collector Stats 패널에 표시

[GitLab](#) · [Docker Compose](#) · [GitHub Actions](#) · [GitLab Runner](#) · [Nginx](#)

온프레미스 Self-Hosted DevOps



구현 언어 / 구성 파일 역할

YAML : Docker Compose, GitHub Actions, GitLab CI/CD, Grafana/Prometheus 설정

Shell : GitLab 운영 점검, 백업·복구 검증, 배포 스크립트

Nginx Conf : Reverse Proxy, HTTPS 내부망 접근 구성 (자체 서명 인증서)

목적

클라우드·SaaS에 의존하지 않고, 개인 Mini PC에서 GitLab 기반 Self-Hosted DevOps 인프라를 구축. 초기 배포 자동화, GitLab Runner 기반 CI/CD, Nginx HTTPS Reverse Proxy, Prometheus/Grafana Monitoring, Backup 기준까지 온 프레미스 운영 흐름으로 검증한 프로젝트.

핵심은 GitLab 서버 실행 자체가 아니라 Bootstrap → 내부 CI/CD → Reverse Proxy → Monitoring → Backup/Recovery 기준까지 이어지는 운영 흐름 재현

자동화 흐름 역할 분리

구분	역할	실행 주체
GitHub Actions	GitLab 서버 초기 배포	self-hosted runner
GitLab Runner	CI/CD Pipeline 실행	Docker Executor

GitHub-hosted runner 한계 확인

GitHub 클라우드 환경에서 실행

→ 내부망 사설 IP Mini PC 접근 불가

→ dial tcp <IP>:22: i/o timeout 발생

해결: Mini PC WSL2에 self-hosted runner 등록

GitHub job을 outbound로 받아 실행하는 구조로 전환

GitLab 리소스 튜닝 (N100 대응)

```
puma worker_processes = 1
sidekiq concurrency = 5
postgresql shared_buffers = 256MB
Docker Desktop Memory 제한
WSL2 내부 파일시스템 기반 Volume
```

Reverse Proxy 구성

- Nginx GitLab 앞단 배치
- gitlab.local 내부망 도메인 (hosts 기반)
- 자체 서명 인증서 HTTPS 내부망 접근 검증
- http://<IP>:8080 복구 경로로 유지

Phase별 구현 및 검증 완료 현황

하드웨어 환경

장비	EcoBe-A1 Mini PC
CPU	Intel N100 (4C/4T, TDP 6W)
Memory	16GB
Host OS	Windows 11
Linux	WSL2 Ubuntu
Runtime	Docker Desktop (WSL2 Integration)

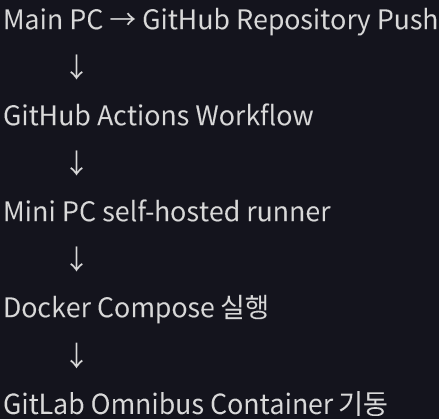
Phase별 설명

단계	내용
Phase 1	Mini PC 기본 환경 구성
	WSL2 Ubuntu 설치, Docker Desktop WSL2 Integration, Main PC → Mini PC SSH 접속 확인 (사설 IP : 172.30.1.67)
Phase 2	GitHub Actions 기반 초기 배포
	GitHub-hosted runner 내부망 접근 한계 확인 후 Mini PC에 self-hosted runner 등록. Docker credential helper 오류 대응. Job deploy Succeeded 확인
Phase 3	GitLab 서버 구축
	GitLab Omnibus Container 구성. /home/user/gitlab 기준 runtime volume 분리. root 관리자 로그인, gitlab-ctl status 주요 서비스 run 확인
Phase 4	GitLab Runner 분리 구성
	Docker Executor 기반 Runner 등록. tag: docker/mini-pc/wsl2. 테스트 Pipeline #1 Passed (약 21초) 확인
Phase 5	CI/CD Pipeline 구성
	validate-compose / deploy-readiness-check Job 구성. CI Job Container 안에서 docker version, docker compose config 검증. Pipeline #4 Passed
Phase 6	운영 안정성 검증
	장기 종료 후 GitLab 재기동. projects/runners/application_settings 유지 여부 확인. db:schema:load 로그 기반 DB 재초기화 판단 기준 수립
Phase 7~9	Nginx SSL · Monitoring · Backup
	gitlab.local HTTPS 접근. Prometheus/Grafana provisioning. cold backup 기준 + DB 재초기화 검증 절차 문서화

GitHub Actions와 GitLab Runner 역할 분리

초기 Bootstrap은 GitHub Actions self-hosted runner가 담당하고, GitLab 구축 이후에는 GitLab Runner가 Repository Pipeline 실행

Bootstrap 흐름



역할 분리 기준

구분	담당 범위
GitHub Actions	GitLab 서버 초기 배포, 내부망 Mini PC에서 Docker Compose 실행
GitLab Runner	GitLab Repository Pipeline 실행, validate/deploy-check Job 검증

분리 이유

- GitLab이 없을 때는 GitLab CI/CD를 사용할 수 없음
- GitHub-hosted runner는 내부망 Mini PC SSH 접근 불가
- Mini PC WSL2 runner는 outbound로 GitHub Job 수신 가능
- 초기 배포와 내부 CI/CD를 분리해 순환 의존 제거

GitHub Actions 배포 Workflow

```
deploy-gitlab.yml
├ GitLab / Runner / Nginx / Monitoring runtime directory 생성
├ gitlab.local 자체 서명 인증서 준비
├ docker-compose.gitlab.yml 배포
├ Grafana datasource/dashboard provisioning 복사
└ Docker Compose 기반 서비스 기동
```

검증 기준

- Mini PC self-hosted runner 상태: Idle / Linux / X64
- GitLab Web UI 접근 가능
- GitLab Container 주요 서비스 run 상태 확인
- 배포 자동화와 GitLab 내부 Pipeline 경로 분리 완료

Docker Executor 기반 Pipeline 검증

Pipeline 구조 (.gitlab-ci.yml 실제 코드)

```
stages: [validate, deploy-check]

default:
  tags: [docker, mini-pc, wsl2]

validate-compose:
  stage: validate
  image: docker:26-cli
  script:
    - docker version
    - docker compose -f infra/compose/
      docker-compose.gitlab.yml config

deploy-readiness-check:
  stage: deploy-check
  image: alpine:3.20
  script:
    - echo "Deploy readiness check"
    - uname -a
    - test -f infra/compose/docker-compose.gitlab.yml
```

GitLab 서버 자체를 Pipeline으로 재배포하면 순환 의존성 발생
→ 현재는 Compose config 검증 · readiness check만 수행

Pipeline 검증 결과 (#4)

[Pipeline #4 Link](#)

항목	결과
Pipeline ID	#4
Status	Passed
Job 수	2
실행 시간	17초
validate-compose	Passed
deploy-readiness-check	Passed

Docker Socket 전달 설정

Runner Container 마운트:

`-v /var/run/docker.sock:/var/run/docker.sock`

config.toml에도 필요:

```
[runners.docker]
volumes = ["/cache",
"/var/run/docker.sock:/var/run/docker.sock"]
```

→ CI Job Container 안에서 Host Docker daemon 접근

Nginx HTTPS 접근과 Grafana Provisioning

Reverse Proxy 접근 흐름

Main PC

↓ https://gitlab.local

Nginx Reverse Proxy

↓ proxy_pass http://gitlab:80

GitLab Omnibus Container

복구 경로 유지

- http://<Mini-PC-IP>:8080 → GitLab Container
- Nginx 구성 기준
- gitlab.local 내부망 도메인 사용
- 자체 서명 인증서로 HTTPS 접근 검증
- HTTP 요청은 HTTPS로 301 redirect
- client_max_body_size 0 설정
- GitLab Container는 내부 Docker network 이름으로 접근

Monitoring Stack

구성	역할
Prometheus	target scrape, 자체 process metrics 수집
Grafana	Prometheus datasource, dashboard 시각화
Provisioning	datasource.yml / dashboards.yml / dashboard JSON

Provisioning 파일 구조

infra/monitoring/grafana/

└ provisioning/datasources/datasource.yml

└ provisioning/dashboards/dashboards.yml

└ dashboards/mini-pc-devops-overview.json

infra/monitoring/prometheus/

└ prometheus.yml

검증 완료

- gitlab.local hosts 해석 확인
- https://gitlab.local GitLab Web UI 접근 확인
- Prometheus target UP 확인
- Grafana Prometheus datasource 자동 연결
- Dashboard provisioning 자동 로드

온프레미스 운영에서 배운 것들

주요 트러블슈팅 4가지

ISSUE 01 : GitHub-hosted runner → Mini PC 내부망 접근 불가

dial tcp <IP>:22: i/o timeout. GitHub 클라우드에서 사설 IP 직접 접근은 불가능.
→ Mini PC WSL2에 self-hosted runner 등록, GitHub job을 outbound로 수신하는 구조로 전환

ISSUE 02 : Docker credential helper 오류

WSL2 환경에서 GitHub Actions runner가 Docker 명령 실행 시 credential helper 관련 오류 발생. → credential helper 설정을 WSL2 환경에 맞게 수정 후 해결

ISSUE 03 : 장기 미가동 후 GitLab DB 재초기화

장기 종료 후 재기동 시 db:schema:load 로그 발생 → 프로젝트·러너·계정 전부 초기화.
WSL2+Docker Desktop 환경의 볼륨 영속성 한계로 판단. → 장기 운영 미확정, cold backup 절차 수립 및 재초기화 판단 기준 문서화

ISSUE 04 : Runner token 불일치 — 403 Forbidden

DB 내 초기화 후 기존 Runner token이 무효화.
GitLab UI에서 새 token 발급 후 gitlab-runner register 재실행으로 해결.

핵심 학습 포인트

- **운영 판단 기준은 수치와 로그로 결정**
Container가 실행 중이라도 DB 내부 상태(projects, runners, settings)로 실제 운영 가능 여부를 판단해야 함
- **부트스트랩 자동화의 실용성**
GitLab이 없을 때 GitHub Actions로, 있을 때 GitLab Runner로. 두 흐름의 역할을 명확히 분리해야 순환 의존 없이 구성 가능
- **온프레미스의 현실**
N100 Mini PC에서 GitLab Omnibus(PostgreSQL, Redis, Gitaly, Rails, Sidekiq)를 모두 올리는 것은 리소스 한계가 존재. 한계 인식 자체가 설계 역량
- **Monitoring은 판단을 위한 데이터**
Prometheus/Grafana 수치보다 "무엇을 수집하고 무엇을 제외할지" 범위 결정이 핵심 설계

운영 금지 명령

- ❌ docker compose down -v
- ❌ docker system prune --volumes
- ❌ rm -rf /home/user/gitlab/data
- ❌ GitLab image latest 사용

- ✅ docker compose stop (볼륨 보존)
- ✅ 재배포 전후 mount 경로 확인
- ✅ image 버전 명시 고정

개선 예정 / 확장 목표

1. Backup 자동화 : 정기 backup script 구성
2. Restore 검증 : 복구 후 projects/runners/settings 확인 절차 표준화
3. Health Check : GitLab/Grafana/Prometheus 상태 점검 스크립트 분리

각각의 프로젝트에서 사용한 기술 전체 목록

BACKEND & APPLICATION

분류	기술	사용 프로젝트
Language	Python 3 / Java 17 / Kotlin	01 / 02 / 03
Framework	FastAPI / Spring Boot 3.4.2	01 / 02
Resilience	Resilience4j (CB, Count-based)	02
Template	Jinja2	01
Build	Gradle	02 · 03

OBSERVABILITY & TESTING

분류	기술	사용 프로젝트
Metrics	Prometheus	02 · 04
Metrics Lib	Micrometer	02
Dashboard	Grafana (provisioning)	03 · 04
Tracing	MDC + TracerIdFilter	02
Load Test	k6, Apache JMeter	03 / 02
System Info	psutil, systemctl	01

DATA & MESSAGING

분류	기술	사용 프로젝트
RDBMS	PostgreSQL 16	02
NoSQL	MongoDB	01
Cache	Redis 7 (L2) + Caffeine (L1)	02
Messaging	Apache Kafka, xk6-output-kafka	03
Time-Series	InfluxDB v3	03

INFRA & DEVOPS

분류	기술	사용 프로젝트
OS	Rocky Linux 9 / Windows 11 + WSL2	01 / 04
Container	Docker, Docker Compose	전체
SCM	GitLab Omnibus	04
CI/CD	GitHub Actions, GitLab Runner	04
Automation	Ansible	01
	GitHub Actions + Docker Compose	04
Proxy	Nginx (Reverse Proxy, SSL)	01 · 04
Secret	Ansible Vault	01
	Spring Cloud Vault / HashiCorp Vault	02

각각의 프로젝트 핵심 키워드 & 성과 요약

프로젝트	핵심 목적	주요 기술	검증 성과	배운 점
01 Server Monitor	OS 환경 차이 직접 검증. 운영 판단형 모니터링	FastAPI · psutil · systemctl · MongoDB · Ansible	Windows → Docker → Native 3단계 이식 검증 완료. Ansible Vault 기반 운영 자동화	같은 코드, 다른 OS에서 다른 동작. 단계별 격리가 원인 특성의 전제
02 Circuit Breaker	극한 부하에서 시스템 반응 정밀 계측. 방어 기제 실증	Spring Boot · Resilience4j · PostgreSQL · Redis · JMeter · Spring Cloud Vault	CB CLOSED → OPEN 상태 전이 실증. Attack/Normal TG 분리 검증. trace_id 전 구간 추적	장애는 설계로 제어. 불변 설정 없이는 실험 재현성 없음
03 k6 Pipeline	실시간 메트릭 스트리밍 파이프라인 구축 + 병목 개선 실증	k6 · Kafka · Kotlin · InfluxDB v3 · Grafana	end-to-end 파이프라인 검증 완료. Kafka lag 83.2% 감소 수치 확보	병목은 단일 변수 변경 + 수치로 증명. 파이프라인 경로 분리가 관측의 전제
04 Self-Hosted DevOps	온프레미스 환경에서 GitLab 기반 DevOps 플랫폼 직접 구축 및 내부망 CI/CD 운영 구조를 검증	GitLab · GitHub Actions · GitLab Runner · Docker Compose · Nginx · Prometheus · Grafana · WSL2	GitHub Actions 기반 초기 배포, GitLab Runner CI/CD, Nginx HTTPS Reverse Proxy, Prometheus/Grafana 모니터링 구성 검증	Self-hosted 운영은 서버 설치뿐 아니라 접근 경로, Runner 실행 위치, 데이터 보존, 관측 기준까지 함께 설계 필요

공통 설계 원칙

직접 검증하고 재현

정상 동작 확인에 그치지 않고 경계 조건·부하 상황·운영 제약을 재현해 시스템 반응을 데이터로 확인

단계별 격리

OS 이식, 부하 증가, 설정 변경 — 항상 단일 변수만 변경해 원인과 결과를 분리

수치로 증명

Kafka lag 83.2% 감소, CB 15건 실패 시 OPEN — 주관이 아닌 측정값과 재현 절차로 결론 도출

한계도 결과물

운영 제약, 미해결 리스크, 재현 조건까지 문서화 — 성공 여부뿐 아니라 한계를 기록하는 것도 검증의 일부

SEO HEE JUNG

Thank you.
For reading.
See you.

저를 더 알고 싶으시다면.  Notion

Thanks for your time.